Developer Handbook – Fighting Engine

1. Introduction

The Fist First Fighting Engine is a fully **JavaScript (ES6)** 2D fighting game that runs inside a single HTML page.
It uses **Canvas 2D**, the **Web Audio API**, and **WebSockets** for networking.
All data (characters, moves, stages, settings) is stored in the global project object.

---

2. Core Concepts

Concept	Description
Game Loop	update() and render() are called every frame via requestAnimationFrame.
Fighters	Two instances of the Fighter class (p1, p2) represent the players.
Projectiles & Mines	Projectile and Mine classes for ranged attacks.
Stages	Contain backgrounds, platforms, falling items, travelcubes, and weather.
Camera	Follows the midpoint of the two fighters with zoom.
Input	Keyboard and gamepad (via navigator.getGamepads).
Network	WebSocket server for online PvP.
Plugins	JavaScript code that can be dynamically loaded and executed.

---

3. The project Object

The central data object holds all configuration and is persisted in localStorage (except images/audio).

javascript

```
project = {
  characters: { /* id → FighterConfig */ },
  stages: [ /* StageConfig */ ],
  settings: { /* all settings */ },
  sprites: { /* id → base64 image */ },
  audio: { /* name → base64 audio */ },
  selectedP1Char: 'fighter',
  selectedP2Char: 'fighter',
  currentStageId: 'dojo',
  laneMode: false,
  cameraZoom: 1.0,
  isPublished: false   // becomes true after publication
}
```

Any change to project is automatically picked up by the engine.

4. The Fighter Class

A Fighter is created with new Fighter(charId, x, facing, playerIdx).
Important properties and methods:

Property	Description
health, maxHealth	Current and maximum health.
energy, maxEnergy	Energy bar (0–100).
speed, gravity	Movement speed and gravity.
state	'idle', 'walk', 'jump', 'crouch', 'attack', 'hit', 'block', 'block_crouch', 'ko', 'swept',
moves	Array of Move objects.
input	Object with booleans: up, down, left, right, forward, back.
facing	1 (right) or -1 (left).
onGround	Boolean.
comboCount, comboTimer	For combo display.

Method	Description
reset(x, facing)	Resets the fighter to starting state.
update(opponent)	Processes input, physics, attacks, blocking, etc.
draw(ctx, camera)	Draws the fighter using sprites or fallback geometry.
takeDamage(dmg, fromDir, blocked)	Applies damage (or heals with negative damage).
startAttack(idx)	Starts an attack from the moves list.
startAttackByCommand(button, history)	Finds a move based on command + input history.

Method	Description
playSound(type)	Plays a sound (attack, damage, block, etc.).
spawnBlood(count)	Generates blood particles.

4.1 Move Object

Each move has the following fields (extended with extra options):

javascript

```
{
  id: 'punch_l',
  name: 'Light',
  damage: 8,
  range: 55,           // reach in pixels
  speed: 12,           // duration in frames (normal) / projectile speed / mine life
  command: '1',        // e.g. 'd+f+1' or 'u+2'
  row: 4,              // sprite row in the spritesheet (normal attacks only)
  startFrame: 0,
  endFrame: 3,
  type: 'normal' | 'projectile' | 'mine' | 'grab' | 'sweep',
  height: 'low' | 'mid' | 'high',
  unblockable: false,
  chipDamage: true,    // whether blocking still causes damage
  energyRequired: 0,   // energy percentage (0–100)
  generatesEnergy: 0,  // energy gained by the attacker on hit
  counterType: 'none' | 'grab' | 'lowmid' | 'highmid',
  throwPower: 8,       // only for type 'grab'
  throwHeight: -10,
  startDistance: 0,    // minimum distance for projectile/mine
  friendlyFire: false, // only for mines
  teleportX: 0,        // target displacement (X)
  teleportY: 0,        // target displacement (Y)
  attackImage: null,   // base64 image shown over the fighter during the attack
```

```
  projectileImage: null,
  mineImage: null
}
```

4.2 Command System

Commands consist of **directions** (f = forward, b = back, d = down, u = up) and a **digit** (1-6) indicating the button.
Examples: '1' (only button 1), 'd+f+1' (quarter circle forward + button 1), 'u+b+2' (quarter circle back + button 2).
The engine keeps a history of the last 12 input events and matches them against the command string.

---

5. Stages and Environment

A stage is defined in project.stages:

javascript

```
{
  id: 'dojo',
  name: 'Dojo',
  bg: 'dojo',          // key for background colours (see BG_COLORS)
  imageData: null,     // base64 image or video (displayed as background)
  audioData: null,     // base64 audio (looped during the stage)
  platforms: [ { x, y, w, h } ],  // spring platforms (only in normal mode)
  itemConfigs: [ { type: 'bomb'|'health', width, height, imageData } ],
  travelcubeConfigs: [ { x, y, width, height, targetStageId, imageData } ],
  colorTheme: 'default' | 'sepia' | 'grayscale' | ...,
  weather: 'none' | 'rain' | 'snow' | 'wind' | 'storm' | 'sandstorm' | 'fog',
  hasSpringPlatform: boolean
}
```

5.1 Platforms

- Active only when laneMode is off.
- Player can stand on them and fall through by pressing down.

5.2 Falling Items

- Items drop randomly from above.
- On contact with a fighter: bomb deals 15% damage, health restores 20%.

5.3 Travelcubes

- Rectangular zones that, when touched by a fighter, **teleport both fighters** to another stage.
- Include a cooldown to prevent infinite loops.

5.4 Weather

- Visual effects: rain, snow, wind, storm, sandstorm, fog.
- Drawn via drawWeather() and updated in updateWeather().

---

6. Settings (project.settings)

These are saved in localStorage under the key 'fighting_settings'.

Important fields:

Field

audioEnabled

roundCount

timerSeconds

aiDifficulty

ringOutEnabled

ringOutLeft, ringOutRight

startX1, startX2

controls

victoryPoints, vpPerWin

showBlockEffect, showCounterShield, showRecoverFlash, showComboText

healthColorP1, healthColorP2, healthBadColor, energyColor, energyFullColor, timerColor, introOutroColor, ri

Field

healthBarHeight, healthBarBorderRadius, energyBarHeight, energyBarBorderRadius

ringOutText, startFightText, roundWinText, roundDrawText, matchWinText

plugins

---

7. Controls

Mappings are stored in project.settings.controls.p1 and .p2.
Each player has keyboard and gamepad mappings.

7.1 Keyboard

An object with actions as keys and keys as values.
Actions: 'left', 'right', 'up', 'down', 'attack1' … 'attack6'.

7.2 Gamepad

An object with actions as keys and a configuration:

- For an axis: { type: 'axis', axis: 0, direction: -1 } (-1 or +1 for negative/positive direction).
- For a button: { type: 'button', index: 0 }.

Remapping is done via the UI (Settings → Controls) or by directly modifying the controls object.

---

8. Networking (WebSocket)

The engine supports **PvP over network** via a **WebSocket server** running on port 8765.

- **Host**: generates a UID and waits for a joiner.
- **Joiner**: enters the UID and server IP to connect.

During the fight, **only input** and **game state** are synchronised.
The engine uses **deterministic logic**, so both clients compute the same outcome.

Key networking functions:

- connectWebSocket(uid, role, ip)
- sendMyInput(input)
- Received messages: 'joined', 'start', 'input', 'state', 'error'.

---

9. Plugins

Plugins are JavaScript functions that are executed after settings are loaded.
They can be added via the Settings modal (upload .js or paste code).

Each plugin object:

javascript

```
{
  name: 'My plugin',
  code: 'console.log("Plugin active!");',
  enabled: true
}
```

Plugins are run via executeAllPlugins().
They have full access to project, p1, p2, camera, and all global functions.

---

10. Export and Import

- **Export**: exportProject() serialises the entire project object into a JSON file.
- **Import**: importProject(fileOrString) loads a JSON file and replaces the current project data.

After import, all characters, stages, sprites, audio, and settings are reloaded.

---

11. UI Extensions

The interface consists of a **top menu** with dropdowns
(modeSelect, actionSelect, utilitySelect).
You can easily add new options by editing the HTML and attaching event listeners.

Key modals:

- **Characters** (openCharacterEditor)
- **Stages** (openStagesEditor)
- **Settings** (openSettingsModal)
- **Sprite Info** (this handbook) (showSpriteInfo)

---

12. Development Tips

1. **Debugging**: use console.log in the update loop or set breakpoints in update().
2. **Adding a new move**: use the character editor or manually push a move to project.characters[id].moves.
3. **Adding a new stage**: use the stage editor or push a new object to project.stages.
4. **Custom sprite sheets**: upload an image in the character editor and set spriteFrameWidth, spriteFrameHeight, cols, and rows.
5. **Background video**: upload an MP4/WEBM in the stage editor (it will play on the canvas).
6. **Network server**: run a WebSocket server on port 8765 (see ws:// in connectWebSocket).
7. **Persistence**: all settings and data are stored in localStorage (except images/audio – they are not persisted, so export your project to preserve them).

---

13. Overview of Key Global Functions

Function	Purpose
fullReset()	Resets the entire match, including scores and timer.
resetRound()	Resets only the current round (keeps scores).
setMode(mode)	Switches between game modes ('pvp', 'pvai', 'training', etc.).
setStage(stageId)	Loads a stage and starts its audio.
update()	Main update for physics, AI, input, projectiles, mines, etc.
render()	Draws everything on the canvas.
pollGamepad()	Reads gamepad input and applies it.
importProject() / exportProject()	Serialisation/deserialisation of project data.
showToast(msg, type)	Displays a brief message at the bottom of the screen.
togglePause()	Pauses/resumes the game.